

Trainer Teaching Guide: Spring Boot Session 4 - Microservices with Spring Cloud

A detailed, textbook-style script for [springboot-session4-theory.html](#). Simple English, examples, analogies and natural fillers - read it aloud and it sounds like teaching, not reading.

Presenter: Purvam Prajapati - Faculty, AHD

Companion to: [springboot-session4-theory.html](#) (22 slides)

HOW TO USE THIS GUIDE:

- ON SCREEN tells you what the slide shows.
- SAY is the full teaching script - read it word-for-word; fillers and questions are built in.
Pause where you see '...'.
- EMPHASISE / DELIVERY TIP are notes for you - do NOT read these aloud.

Slide 1 - Title: Microservices with Spring Cloud

ON SCREEN

The blue cover slide marking the final session: Service Registry, API Gateway & Load Balancing.

SAY (read this aloud)

Welcome, everyone, to the final session of our Spring Boot series. Take a second to appreciate how far you have come - you started by printing 'hello', and today you are going to build a real, distributed system.

Session 3 ended with the idea of microservices - the concept of breaking a big app into many small services. Today, we stop talking about the idea and actually build the machinery that runs them. By the end of this session, you will have four separate Spring Boot apps running on your machine at the same time, all finding each other and sharing traffic automatically.

Now, fair warning - this is the most infrastructure-heavy session of the four. There are a few more moving parts than usual. So the theory is loaded right at the front, and every single concept I teach maps directly to something you will build in the next forty-five minutes. So stay with me through the theory, because it is the map for the whole lab.

DELIVERY TIP

Acknowledge the difficulty openly - 'this one has more moving parts'. Naming it upfront lowers anxiety. Keep pointing back to slide 14, the architecture flow, as the anchor throughout.

Slide 2 - Agenda

ON SCREEN

Six agenda items: monolith vs microservices recap, the design patterns, Spring Cloud, architecture flow, Eureka, API Gateway.

SAY (read this aloud)

Here is the plan. First, a quick sixty-second refresher on monolith versus microservices, just to anchor it. Then the five classic design patterns of microservices. Then Spring Cloud - the toolbox that gives us those patterns. Then the full architecture flow, which is the mental model for the entire session - I will keep coming back to it. Then Eureka, which is the service registry, with its server and its clients. And finally the API Gateway, with routing and load balancing.

So the big idea to hold onto from the start: keep that architecture diagram - slide fourteen - in your mind the whole time. Everything we build is a piece of that one picture. Let's begin.

Slide 3 - Session 3 recap

ON SCREEN

Three click-to-reveal flash cards: what a DTO does, @ResponseStatus, and what Spring Security adds.

SAY (read this aloud)

Quick warm-up, three questions - answer in your head, then reveal.

Question one: what is a DTO and why use it? Think... Reveal. A plain class that carries only the data the caller needs - it hides sensitive fields, shapes the response, and keeps your API independent of the database.

Question two: what does @ResponseStatus do on a custom exception? Think... Reveal. It tells Spring which status code to send when that exception is thrown - so our not-found exception automatically produces a clean 404.

And question three: what does Spring Security do the moment you add it? Reveal. It locks down everything - a login page appears, a random password is printed in the console, and the default username is 'user' - all before you write any code.

Great. That is the polished, professional single service we built. Today, we run several of them together.

Part 1 - Concepts & Patterns

Slide 4 - Monolith vs microservices, revisited

ON SCREEN

A two-column recap: monolith versus microservices, with a key box on today's framing.

SAY (read this aloud)

A quick sixty-second anchor before we dive in. Remember, last session we talked about WHY microservices exist. Today we build the infrastructure that runs them. So let me just refresh the contrast.

A monolith is one single deployable app, with one shared database. Its weaknesses: one failure can bring the whole thing down, and to scale it you must duplicate the entire app. Microservices, on the other hand, are many independent services, each with its own database. Their strengths: if one fails, the others keep going, and you can scale just the busy one.

Now here is today's framing, and this is the important bit. Microservices give us independence and scalability - wonderful. But they introduce a brand-new problem that a monolith never had. When you have many separate services, scattered around, starting and stopping... how on earth do they find each other and talk to each other? That question is exactly what the rest of this session answers.

EMPHASISE

This framing - 'microservices create a new find-each-other problem' - is the thread for the whole session. Everything that follows (registry, gateway) is a solution to it. Say it deliberately.

Slide 5 - Horizontal vs vertical scaling, revisited

ON SCREEN

Two columns - vertical and horizontal scaling - and a warning box introducing the two new problems.

SAY (read this aloud)

A quick reminder of scaling from last time. Vertical scaling - scale up - is making one machine more powerful; simple, but it has a ceiling and gets expensive. Horizontal scaling - scale out - is adding more machines, each running a copy; no ceiling, cost-effective, and fault-tolerant.

Now here is the catch that sets up everything today. When you scale horizontally, you end up with multiple copies of the same service - for example, our Student service running on three different ports at once. And that creates two brand-new headaches. First - how does anyone know all those addresses? Copies come and go, on changing ports. You cannot just hard-code an address. And second - how do we spread the traffic evenly across all those copies?

Hold those two questions in your head. Because the next two patterns - a Service Registry and an API Gateway - are precisely the answers. One solves 'where is everything', the other solves 'share the traffic'. Everything today flows from this slide.

Slide 6 - The five microservices design patterns

ON SCREEN

Five cards: Config Server, Service Registry, API Gateway, Distributed Tracing, Circuit Breaker.

SAY (read this aloud)

Across every real microservices system, the same five patterns show up. Each one solves a specific headache. We will actually build the first two today, and just understand the other three - you will meet all of them on real projects. Let me give you each one with its problem and its fix.

Config Server - the problem: ten services means ten separate config files to manage. The fix: one central, shared config. Service Registry - the problem: services start on changing ports and you cannot hard-code addresses. The fix: a live phonebook of what is running where. API Gateway - the problem: callers should not need to know all your internal addresses. The fix: one single front door. Distributed Tracing - the problem: a request hops through five services and one is slow, but which? The fix: end-to-end tracking. And Circuit Breaker - the problem: service B is down and service A keeps hammering it. The fix: detect the failure, stop calling, return a fallback.

So today we build the Service Registry and the API Gateway. The other three, we will understand by concept. Knowing all five, and what each solves, is what matters.

Slide 7 - Config Server

ON SCREEN

A diagram: a Git repo served by Spring Cloud Config Server, read by Services A, B, and C.

SAY (read this aloud)

Pattern one - the Config Server. Concept only today, but a useful one. Picture this problem: you have ten microservices, and each has its own application dot properties file. Now your database password changes. Ugh - you have to edit ten files and redeploy ten services. Painful and error-prone.

The Config Server fixes this elegantly. Instead of each service keeping its own config, all the configuration lives in one central place - usually a Git repository, so it is version-controlled. A Config Server reads from there, and every service asks the Config Server for its settings when it starts up. Follow the diagram - Git at the top, the Config Server in the middle, and all the services pulling their settings from it.

So now, change that database password in one place, and every service can pick it up - no editing ten files. When would you use it? When you have many services across different environments, like development and production. It is like a company noticeboard - update it once, and everyone reads the same latest information. We are not building it today, but you will definitely meet it.

Slide 8 - Service Registry (Eureka), how it works

ON SCREEN

A four-step flow: Eureka starts, Student MS registers, Customer MS registers, the Gateway asks Eureka. A key box on heartbeats.

SAY (read this aloud)

Pattern two - the Service Registry. This one we build today, and it is the answer to our 'how does anyone find anything' problem. A service registry is simply a live phonebook of every running service - its name, its address, its port. The tool we use is called Netflix Eureka.

Here is how it works, in four steps - follow along. Step one, Eureka itself starts up, on port 8761 - that is the empty phonebook, ready for entries. Step two, our Student service starts, and it announces itself to Eureka: 'Hi, I am STUDENT-MS, I am at port 8080.' Eureka writes that down. Step three, our Customer service starts and registers itself the same way. Now Eureka knows about both. And step four, when the gateway needs the Student service, it just asks Eureka, 'where is STUDENT-MS?' and Eureka hands back the list of live addresses.

So nobody hard-codes addresses anymore - they just ask the phonebook. Beautiful.

EMPHASISE

Highlight the heartbeat - it is the self-healing magic in the key box. Registered services 'ping' Eureka regularly, like saying 'still alive, still alive'. If a service crashes and the pings stop, Eureka removes it from the phonebook after a short wait. So the gateway never sends traffic to a dead service. No human has to clean up. That is automatic self-healing.

Slide 9 - API Gateway, what it does

ON SCREEN

Three cards - routing, load balancing, single entry point - and a sample gateway route in properties.

SAY (read this aloud)

Pattern three - the API Gateway. This is the single front door to your whole system, and we build it today too. The idea: callers talk to just one address, the gateway, and the gateway figures out the rest. It has three jobs - on the cards.

Job one, routing - it looks at the incoming address and forwards it to the right service. Anything starting with slash-students-api goes to the Student service; slash-customers-api goes to the Customer service. Job two, load balancing - if a service has several copies running, the gateway spreads requests across them so none gets overwhelmed. And job three, single entry point - callers only ever need to know one address, the gateway's. All the messy internal addresses stay hidden behind it.

Think of it like the reception desk of a big office building. Visitors do not wander the halls hunting for the right room - they go to one reception desk, and reception directs them. Look at the sample route on screen - an id, a destination written as lb-colon-slash-slash STUDENT-MS, and a path rule. You will write exactly this in the lab.

Slide 10 - Load balancing: Round Robin

ON SCREEN

A visual of four requests cycling across instances 8080, 8082, 8083, then back to 8080. A key box on the lb:// prefix.

SAY (read this aloud)

Let's look at how the gateway actually shares traffic across several copies. The simplest and most common method is called Round Robin, and it is exactly as simple as it sounds - it just takes turns.

Watch the visual. Request one goes to the copy on port 8080. Request two goes to 8082. Request three goes to 8083. And request four loops back around to 8080 again. Round and round, one after another, evenly. It is just like a teacher handing out papers row by row, then starting again at the first row. Fair and simple.

And here is the little piece of magic that switches this on. In your gateway route, you write the destination as lb-colon-slash-slash, then the service name. That lb stands for load balancer. It tells Spring Cloud: 'do not use a fixed address - instead, ask Eureka for all the live copies of this service, and take turns across them automatically.' So those two little letters, lb, are doing a lot of work. You will see this rotation happen live in the lab, and it is genuinely satisfying to watch.

Slide 11 - Distributed Tracing

ON SCREEN

A request flowing through Gateway, Order MS, Inventory MS, Payment MS, and a Zipkin-style timing breakdown.

SAY (read this aloud)

Pattern four - distributed tracing. Concept only, but really important to understand. Here is the problem it solves. In a microservices system, one user request might travel through five different services to get its job done. Now suppose the whole thing feels slow. Which service is the culprit? In a monolith you could just check one log. But here the request is scattered across five separate programs. How do you follow it?

Distributed tracing is the answer. Every incoming request is given a unique trace ID - like a tracking number on a parcel. That same ID is passed along to every service the request touches. Each service logs its work with that ID. Then a tool called Zipkin gathers all the logs that share that ID and stitches them into one timeline.

Look at the breakdown on screen - it shows how long each service took, and immediately you can see the slow one lit up. So just like tracking a parcel across multiple delivery hubs, you can see exactly where the request spent its time, and where it got stuck. The tools are Micrometer and Zipkin. We are not building it today, but the moment you have more than two services, you will be grateful it exists.

Slide 12 - Circuit Breaker

ON SCREEN

A three-state diagram: CLOSED, OPEN, HALF-OPEN, and the transitions between them.

SAY (read this aloud)

Pattern five - the circuit breaker. Concept only, and it borrows its name from the electrical switch in your home. Here is the problem. Suppose service A keeps calling service B, but service B has crashed. If A keeps hammering away at a dead B, it wastes resources, piles up, and the failure can spread - one dead service drags down others. We call that a cascading failure.

A circuit breaker prevents this, with three states - follow the diagram. CLOSED is normal - calls flow through, and it quietly counts failures. If too many calls fail, it trips to OPEN - and now, instead of hammering the dead service, it instantly returns a fallback, like 'sorry, try again later'. No more pointless calls. Then, after a short wait, it goes HALF-OPEN - it cautiously lets one test call through. If that works, great, back to CLOSED, normal service. If it fails, back to OPEN, keep waiting.

It is exactly like the breaker in your house - when there is a problem, it trips to protect the system, and then you carefully check whether it is safe to switch back on. The Spring tool for this is called Resilience4j. Not building it today, but a lovely concept to carry with you.

Slide 13 - Spring Cloud, the toolbox

ON SCREEN

A table mapping each pattern to its Spring Cloud tool, with a 'today?' column.

SAY (read this aloud)

So we have these five patterns - now, what do we actually use to build them? The answer is Spring Cloud. And here is the key thing to understand: Spring Cloud is not one single thing. It is a collection - a toolbox - of separate tools, and you pick the ones you need. Let me map each pattern to its tool, using the table.

Service Registry - the tool is Netflix Eureka; we build this today. API Gateway - the tool is Spring Cloud Gateway; building today. Load Balancing - Spring Cloud LoadBalancer, which we get through that lb prefix; today. Config Server - Spring Cloud Config; concept only. Circuit Breaker - Resilience4j; concept. And Distributed Tracing - Micrometer plus Zipkin; concept.

So the point is - you do not use every tool in every project. You open the toolbox and grab what your system needs. Today we grab three: Eureka, Gateway, and LoadBalancer. And honestly, even just that trio gives you a genuine, production-shaped microservices skeleton. It is like a toolbox at home - you do not use every tool for every job, you pick the right ones.

Part 2 - The Architecture & What We Build

Slide 14 - Full architecture flow

ON SCREEN

The centrepiece diagram: client to API Gateway (9393), which queries Eureka (8761), then routes to Student MS instances and Customer MS.

SAY (read this aloud)

Okay - this is the centrepiece slide, the mental model for the entire session. I will keep pointing back to it during the lab, so let's trace one request through the whole system, step by step. Follow the arrows with me.

It starts with a client - a browser, or our testing tool - and the client knows only ONE address: the gateway, on port 9393. The client sends its request there. The gateway receives it and thinks, 'this is for the Student service - but where is the Student service right now?' So it turns to Eureka, the phonebook on port 8761, and asks, 'where is STUDENT-MS?' Eureka replies with the list of live copies. The gateway then picks one copy using round robin, and forwards the request to it - say, the Student service on port 8080. That service does its work and sends the answer back, and it travels all the way back to the client.

And here is the beautiful part - the client had no idea there were three copies of the Student service. It just talked to one front door. All that finding and sharing happened invisibly behind the scenes.

EMPHASISE

Read it as one simple sentence they can repeat: client hits the gateway; gateway asks Eureka where the service is; Eureka answers; gateway picks one copy and forwards; the response comes back. Pin this diagram up and refer to it at every lab step - it keeps everyone oriented.

Slide 15 - What we're building today

ON SCREEN

Four project cards: Eureka Server, Student MS, Customer MS, API Gateway, each with its port and key dependency.

SAY (read this aloud)

Let me show you the actual shopping list for the lab - the four separate projects you will create. Don't worry, they are each small.

Project one - the Eureka Server, on port 8761. Its job is to be the phonebook. The key ingredient is the Eureka Server dependency, and one annotation, `@EnableEurekaServer`. Project two - the Student service, on ports 8080, 8082, and 8083, because we will run three copies. It uses the Web and Eureka Client ingredients, and registers itself as STUDENT-MS. Project three - the Customer service, on port 8081, built the same way, registered as CUSTOMER-MS. And project four - the API Gateway, on port 9393, the single front door, using the Gateway and Eureka Client ingredients, and it is pure configuration - no controller.

So four little apps, working together. This card is your build checklist - keep it handy during the lab.

DELIVERY TIP

Warn them now about the practical reality, as the slide notes: they will run four - then six - Java apps at the same time, which needs decent RAM. If a machine struggles, it is fine to stop the extra Student copies after they have seen them register. Better to set this expectation before frustration hits.

Slide 16 - Eureka Server setup

ON SCREEN

The main class with `@EnableEurekaServer`, and the four key `application.properties` lines.

SAY (read this aloud)

Let's look at how little it takes to build the Eureka Server - the phonebook. It is almost entirely configuration. On the main class, you add one annotation - `@EnableEurekaServer`. That single line turns an ordinary Spring Boot app into a registry. That is it for the code.

Then four lines in `application dot properties`. Port 8761 - the standard Eureka port that everyone expects. A name for the app. And then two lines that look a bit odd at first - `register-with-eureka` set to false, and `fetch-registry` set to false. Let me explain those, because students always ask.

Here is the thing - normally, a Eureka component acts as a client, registering itself. But the SERVER is the phonebook itself. It does not need to register in its own phonebook - that would be silly. And it does not need to download a phonebook from somewhere else - it IS the phonebook. So we switch both of those off. If you forget these two lines, you will see noisy startup errors as the

server tries to phone a registry that does not exist. So just remember - on the server, both of those are false.

Slide 17 - Eureka Client setup

ON SCREEN

The three key application.properties lines for a microservice, and a key box on the service name.

SAY (read this aloud)

Now, how does a normal microservice join the phonebook? Even easier - just add the Eureka Client ingredient and three properties. And here is a nice modern touch - you do not even need a special annotation anymore; Spring detects it automatically from the ingredient. Older versions needed `@EnableEurekaClient`, so you might see that in old code, but today it is automatic.

The three lines: a port, like 8080, which we will override to run extra copies. A name - `spring dot application dot name` - set to `STUDENT-MS`. And the address of Eureka, so the service knows where to register.

Now, of those three, one line is the most important in the whole lab, so let me flag it strongly.

EMPHASISE

Stress the service name - it is the line that breaks everything if wrong. `spring dot application dot name`, set to `STUDENT-MS`, is the exact name Eureka lists it under AND the exact name the gateway uses to route to it. If there is a typo, or the case is different, the gateway route silently fails and nothing works - with no obvious error. So type it carefully, uppercase, and make sure it matches everywhere.

Slide 18 - API Gateway: manual routes vs discovery locator

ON SCREEN

Two panels: manual routing config versus discovery-locator config, with pros and cons.

SAY (read this aloud)

There are two ways to tell the gateway how to route requests, and you will try both in the lab, so let me compare them.

The first, on the left, is manual routing. Here you spell out each route yourself - you give it an id, a destination written as `lb-colon-slash-slash` plus the service name, and a path rule. The upside: you get clean, custom addresses, like `slash-students-api`, and full control. The downside: more configuration to write, one block per service.

The second, on the right, is the discovery locator. Here you just switch on one setting, and the gateway automatically creates routes for every service in Eureka, using their names. The upside: almost zero configuration. The downside: the service name shows up in the address, like `slash-student-ms`, which is less pretty and exposes your internal names.

So which to use? Manual routes for a polished, production API where you care about clean addresses. Discovery locator for quick prototyping and exploring, when you just want it working fast. The trade-off, in one line, is clean addresses versus zero configuration.

Slide 19 - Running multiple instances

ON SCREEN

Two columns - IntelliJ and Eclipse steps - and a mock Eureka dashboard showing STUDENT-MS with three instances.

SAY (read this aloud)

Now, to actually SEE load balancing, we need more than one copy of the Student service running. And here is the neat part - you do not create a new project for each copy. You run the very same project again, just on a different port, using a small startup setting.

In IntelliJ, you copy the run configuration and add a VM option - dash-D-server-dot-port equals 8082 - and again for 8083. In Eclipse, you do a Maven run with the same VM argument. There is also a command-line way on the sheet. Either way, you are launching the same code on a fresh port each time.

Then you refresh the Eureka dashboard, and - look at the mock on screen - the Student service now shows three copies, on 8080, 8082, and 8083, all registered. It is like opening three identical checkout counters in the same shop.

EMPHASISE

Pre-empt the most common stumble: if all three copies show the same port, the VM argument was not applied. Check it is in the VM OPTIONS box, not the program arguments box, and that it starts with dash-D. This trips people up every single time, so call it out before they hit it.

Recap & Close

Slide 20 - The full journey, Sessions 1 to 4

ON SCREEN

A vertical timeline: Session 1 single controller, Session 2 REST API, Session 3 layers, Session 4 microservices.

SAY (read this aloud)

Let's step back for a moment and look at the whole journey, because it is genuinely something to be proud of. Look at this timeline.

Session one - you built a single Spring Boot app with one controller. The absolute basics - getting something to run. Session two - you turned it into a full REST API, with create, read, update, delete, and JSON. Session three - you made it professional and layered, with DTOs, proper layers, clean error handling, and a real database. And session four - today - you scaled it into a full microservices system, with a registry, a gateway, multiple copies, and load balancing.

Each session built directly on the last. So think about where you started and where you are now: you went from printing 'hello' on a screen, to building a distributed microservices architecture - in just four sessions. That is a huge leap, and what you can build now is a genuine, production-shaped skeleton that real companies run. Really, very well done.

Slide 21 - What's next: the industry roadmap

ON SCREEN

Six cards: Config Server, JWT security, Docker, Kubernetes, CI/CD, tracing with Zipkin.

SAY (read this aloud)

What you have built is the skeleton. A real production system layers more on top, and I want you to see the road ahead - not as pressure, just as a map of where you can go next.

A Config Server, for that centralised configuration we discussed. JWT security, enforced at the gateway and shared across services, so logins work across the whole system. Docker, which packages each service into a neat container so it runs the same everywhere. Kubernetes, which manages and scales and heals all those containers automatically. CI/CD pipelines, which automate building, testing, and deploying each service. And distributed tracing with Zipkin, for that end-to-end visibility.

Now here is the encouraging part. Every single one of those advanced topics sits on top of exactly what you learned in these four sessions. You have built the foundation - and the foundation is the part that takes real understanding. The rest is layering on tools. So you are genuinely ready to keep climbing.

Slide 22 - Closing

ON SCREEN

The closing blue slide marking the end of the series: 'Let's build it.'

SAY (read this aloud)

And that brings us to the end of the theory, and the end of our series together. Let me leave you with this one thought. You started this journey with a single Hello World controller. You are finishing it by standing up a microservices system - with a registry, a gateway, multiple copies of a service, and automatic load balancing. That is a remarkable distance to travel.

So now, one last time - let's open the lab and build it. We will start Eureka, register two services, scale one of them to three copies, and route everything through a gateway with live load balancing that you will watch happen in real time. Let's go.

And genuinely - thank you, all of you, for your focus and your effort across these four sessions. It has been a pleasure teaching you.

DELIVERY TIP

During the lab, keep Eureka running on your own machine as a safety net so the live demo never breaks. And keep slide 14, the architecture flow, on screen throughout the hands-on section - it is the map that keeps everyone oriented.